S20 复盘 · chassis_bridge 节点实现

> 阶段 4（ROS 接口层）交付物回顾 完成日期：2026-10-07 验收结果：`无硬件自测/verify_e2e.sh` → **通过 21 / 失败 0 / 跳过 0**

一、交付了什么

项目	内容
主文件	`src/navbot_bridge/navbot_bridge/chassis_bridge.py`（448 行）
上行链路	O 帧（11 字段，50Hz）→ `/odom` + TF `odom→base_footprint` + `/joint_states`
下行链路	`/cmd_vel` → 差速逆解 → V 帧（4 字段，20Hz）
并发模型	1 个读串口线程（daemon）+ 1 个 50Hz 发布定时器
异常处理	三层：连接层重连 / 数据层静默丢弃 / 时效层暂停发布

e2e 实测数据（2026-10-07 22:45）

检查项	实测	判据
`/odom` 频率	**49.923 Hz**	40~60

检查项	实测	判据
`header.frame_id`	`odom`	一字不差
`child_frame_id`	`base_footprint`	一字不差
TF `odom→base_footprint`	存在	—
`/joint_states`	有数据	—
20% 乱码注入后进程	存活	—
乱码注入后 `/odom`	仍在发布	—
桥日志总行数（注入期间）	**2 行**	< 500

二、三个设计决策（← 这部分是面试会问的）

决策 1：为什么读串口必须独立线程

结论：`ser.readline()` 是阻塞调用，放进 spin 线程等于把整台车的大脑掐停。

实证依据（本机 2026-09-29 实测）：单线程执行器里，定时器第 3 个 tick 的回调 `sleep(6)` →

- 12 个周期凭空消失（tick #3 → 6 秒空白 → tick #4）
- `ros2 service call /ping` 卡 **5.124270576 秒**
- Ctrl+C 退不掉

把 `sleep` 换成 `readline()` 是一模一样的后果。**单线程执行器的模型是「一个队列，一条线程，串行处理所有回调」**——队里有一个人卡住，后面全堵着。

落地方式：

> 读线程（daemon）：只搬运字节 → _on_line()，不解析、不发话题
> 发布定时器（spin 线程）：只取最新值 → 算 → 发，绝不碰串口
> cmd_vel 回调（spin 线程）：组帧 → write 一发即走，不 sleep/不重试

关键细节：`serial.Serial(..., timeout=0.1)` 而不是 `None`。`None` 意味着读不满一行就**永久阻塞**，`self._closing` 永远检查不到，Ctrl+C 直接卡死。（`fake_chassis.py` 里那句「真机上这就是『读串口必须另开线程』的根因」说的就是它。）

决策 2：为什么用「定时器 + 最新值」而不是「收到即发」

结论：读线程的节奏由串口决定（时快时慢），发布节奏必须等间隔，两者只能解耦。

收到即发的问题：串口来 3 帧挤在 5ms 内 → 发 3 条；然后 80ms 没数据 → 一条不发。下游拿到「脉冲式」的 `/odom`，`ros2 topic hz` 显示抖动平均值，Nav2 的传感器融合也会因时间戳不均匀而变差。

定时器 + 最新值：定时器每 20ms 雷打不动取一次发一次。来 3 帧只发最新那帧（旧的直接丢——里程计只关心现在在哪）。

实测验证：`49.923 Hz`，稳稳落在 40~60 判据内。

共享变量必须加锁：`self._latest` 是读线程写、定时器读。`self._latest = frame` 和 `self._latest_recv_t = now` 是**两条语句**，中间可能被切走 → 定时器读到「新 frame + 旧时间戳」→ 算出往前的 dt → 位姿跳一下。**取快照要短**：锁里只做赋值，拿到就出锁，绝不在锁里 publish。

决策 3：里程计积分的三条路，为什么选编码器增量

B1 路线图列出了三条可选路径：

路径	评价
(a) 编码器增量积分	★ 本实现选择
(b) `vel_l`/`vel_r` 反馈 × dt	可用，但反馈值经过 MCU 滤波，丢失细节
(c) 把 vel 当瞬时速度、位姿不积分	✗ 位姿永远是 0，RViz 里车不动

选 (a) 的理由：

1. **最接近真实里程计**——编码器是物理量，直接反映轮子实际转了多少
2. **能反映打滑**——轮子空转时编码器还在涨，`vel` 反馈也一样，但增量的口径最原始
3. **与固件侧口径一致**——`enc_l`/`enc_r` 是 MCU 直接上报的累计值，少一层解释

积分用中点法：

```
yaw_mid = self._yaw + d_yaw / 2.0          # ← 关键
self._x += d_center * math.cos(yaw_mid)
self._y += d_center * math.sin(yaw_mid)
self._yaw += d_yaw
```

直行时中点法与起始法等价；**原地转 90° 时用起始 yaw 会让位移方向差 90°**， 中点法把误差摊一半。对解析解实测：中点法 3.999 mm vs 起始法 5.189 mm。

三、三层异常处理（R3）

三种故障的**处理方式完全不同**，因为危害等级完全不同：

层	触发条件	危害	正确反应	日志策略
连接层	打不开 / 拔线	无数据，但节点还活着	重连 + 节流 warn	5 秒最多一条
数据层	字段数≠11 / 头≠'O' / `int()` 失败	只有这一帧是垃圾	静默丢弃 + 计数	**一条都不打**
时效层	seq 长时间不增长	**下游拿旧值当真值**	暂停发布 + 只警告一次	一次故障一句

日志阶梯是关键：越底层的故障越频繁，就必须越安静。

- 数据层最频繁（≤20% 乱码）→ 只进 `self._bad_frames`，**零输出**
- 连接层中等 → warn 但节流 5 秒
- 时效层最稀有 → `self._stale_warned` 布尔锁死

实测：连接层节流

`_e2e_logs/bridge.log` 里，假底盘被杀后桥持续尝试重连：

```
[WARN] 超过 0.5s 没有新帧，暂停发布 /odom          ← 时效层，只警告一次
[WARN] 串口读取出错，将重连: device reports readiness to read but returned no data
[WARN] 打开串口 /tmp/ttyV1 失败，将重试: [Errno 2] No such file or directory
[WARN] 打开串口 /tmp/ttyV1 失败，将重试: ...        ← 之后每 5 秒一条
```

实测节流间隔：**5.007 / 5.006 / 5.007 / 5.009 / 5.007 秒**，精确锁定 `WARN_THROTTLE_S = 5.0`。

★ 这段日志同时验证了三层的分工： - 「暂停发布 /odom」只出现一次 → 时效层布尔锁生效 - 「读取出错，将重连」出现一次 → 异常落到了外层重连分支，节点没崩 - 「打开失败，将重试」按 5 秒节流 → 连接层节流生效

如果不节流会怎样：`timeout=0.1` 的 readline 在设备消失时会飞快抛异常，一秒能刷几十条 → 第 6 组判据（日志 < 500 行）直接红。

为什么时效层必须「暂停发布」而不是「继续发旧值」

这是最反直觉的一条。

车停了、MCU 死机了，`self._latest` 里还留着最后那帧位姿。**如果继续发**：

- Nav2 拿到一份**不变但一直刷新**的位姿
- AMCL 把它当观测值反复校正
- 代价地图围绕这个「假真值」不断滚动
- 现象：**车静止不动，RViz 里地图却在缓慢漂移、路径规划开始乱跳**

因为「车没动、话题也在正常发」，你不会怀疑到桥身上。

正确做法是沉默：`/odom` 断流 → Nav2 的 `controller_server` 因 odom 超时停发 `/cmd_vel` → MCU 侧 500ms 失联停车兜底接上，整条链路优雅降级。

`_stale_warned` 的复位时机：`_on_line` 里收到好帧要 `self._stale_warned = False`。不复位的话，**下一次再断流就再也不报警**—— 故障静默失败，比没有告警更糟。

四、这次踩的坑与教训

坑 1：install 落后于 src（改了源码没重新 build）

现象：补完 R1/R2/R3 后跑 e2e，第 5 组「桥启动后立刻退出」：

```
File ".../install/navbot_bridge/lib/python3.10/site-packages/navbot_bridge/
chassis_bridge.py", line 314
    raise NotImplementedError("R2 未实现：定时器 + 最新值发布")
```

但此时源码里 `_on_publish` 已是完整实现，**且同一个方法在源码里是 274 行，不是 314 行**。

根因：ROS2 里 **`src/` 不是运行时的代码**。`colcon build` 默认是 copy 模式， 改完 `src/` 不重新 build，`ros2 run` 加载的还是上一次构建的副本。

识别信号：**报错行号与源码对不上** + 报的是源码里已删掉的代码。 这两个信号一出现，立刻 `diff` 源码与实际加载的文件，不要怀疑「代码没保存」。

修复：

```
colcon build --packages-select navbot_bridge --symlink-install
```

`--symlink-install` 后 `build/` 里变成指向 `src/` 的符号链接， **以后改 `.py` 立即生效，不用再 build**（改了 `setup.py`/新增文件仍需重建）。

坑 2：`--symlink-install` 让 install 结构完全变了

切到 symlink 模式后，install 里的结构变成：

```
install/navbot_bridge/lib/python3.10/site-packages/
  └── navbot-bridge.egg-link        ← 指向 build/navbot_bridge
      ★ 该目录下**没有任何 .py 文件**

build/navbot_bridge/
  └── navbot_bridge -> ../../src/navbot_bridge/navbot_bridge   ← 符号链接
```

检查「源码是否生效」时，永远不要猜 install 路径。 正确做法是问 Python：

```
spec = importlib.util.find_spec('navbot_bridge.chassis_bridge')
print(os.path.realpath(spec.origin))
```

`realpath` 会把 symlink 模式的 `build/` 路径解析回 `src/`， 两者归一化后是同一个文件 → 判据天然通过。

教训：硬编码路径的检查逻辑，会在切换构建模式时变成**假失败**—— 比没有检查更糟，因为它把「环境正常」误判成「环境坏了」。 **永远用询问代替假设。**

坑 3（测试侧）：不要拿模拟器真值当基准

写端到端积分仿真时，第一版拿 `DiffDriveSim` 的内部位姿当真值比， 结果「中点法」输给了「起始法」。查下去发现：

`DiffDriveSim.step()` 内部也是用欧拉积分（起始 yaw）：

```
self.x += v * math.cos(self.yaw) * dt      # ← 起始 yaw
self.yaw += omega * dt
```

拿中点法去比它的真值 = **两套积分法互比**，谁跟模拟器用同一种方法谁就赢。

正确做法：跟**解析解**比（圆的参数方程），实测吻合 1.9 mm。

五、意外收获：量化误差才是里程计误差主因

对拍过程中挖出的最有价值的一条结论。

决定性实验：圆弧 `v=0.2 m/s, ω=0.5 rad/s` 跑 4 秒，只换编码器输入类型：

编码器输入	yaw 误差	位置误差
整数脉冲（真机情况）	**+0.04203 rad**	15.27 mm
浮点脉冲（去掉量化）	**−0.00200 rad**	2.67 mm

误差掉 20 倍 → 量化误差是主导，积分算法本身没问题。

量化误差的理论界：

```
圆弧场景 v=0.2 ω=0.5 → v_l=160 mm/s, v_r=240 mm/s
每周期脉冲 = 33.0964 (左) / 49.6446 (右) → 取整误差各 ≤ 0.5 脉冲/拍
dω 误差  ≤ (0.5 + 0.5) × m_per_pulse / L / dt
         = 1 × 9.6687e-5 / 0.16 / 0.02
         = 0.0302 rad/s
4 秒累计 ≤ 0.121 rad; 实测 0.042 rad 在界内
```

这条的实际意义：

- 里程计漂移**不是**「算法不够好」，而是**整数编码器的物理事实**

- 真机表现：车开回原点，odom 显示偏了 1~2 厘米
- 修漂移靠 `/scan` + AMCL，**不是**在桥里能解决的
- 桥的任务是把漂移**控制在合理范围**，不是消除它

验收时的用法：定阈值必须按量化理论上界给。 拍脑袋定「误差 < 2%」或「< 10 mm」会把正常行为误判成 bug。

六、遗留问题与下一步

遗留 1：`/joint_states` 的 uint16 回绕（★ 建议真机前处理）

`/joint_states.position` 用 `enc_l`/`enc_r` 的**绝对值**换算， 但协议里这两个字段是 **uint16**（0~65535）。

真机跑长距离会回绕 → 关节角突跳 → RViz 里轮子瞬移。 按 `10342.6 脉冲/米` 算，约 **6.34 米**就绕一圈。

为什么 e2e 测不出来：假底盘单次运行跑不到 6 米，不触发。 典型的「验收通过但真机会踩」。

可选的修法： - 固件侧把 `enc_l`/`enc_r` 改成 32 位累计（推荐，改协议） - 或桥侧自己维护回绕检测（差分时判断跳变超过 32768 就补偿 65536）

遗留 2：`deadband_pulses` 死区未实现

B1 建议声明 `deadband_pulses=2`，但骨架文件没声明、我按骨架实现也没加。

为什么需要：静止时编码器抖动 ±2 个脉冲会算出 **9.67 mm/s** 的假速度 （`test_protocol.py:277` 那条测试就是验这个）。不打死区， 车静止时 `/odom.twist` 会显示约 1 cm/s 的噪声速度。

影响：AMCL 会被这点速度噪声轻微干扰，但不致命。 **建议**：真机调试阶段加上，用 `abs(d_enc) < deadband_pulses → 视为 0`。

遗留 3：`/joint_states` 的轮子转向未在 RViz 目视验证

`hz` 只证明「有数据、频率对」，**不证明「方向对」**。 前进时两轮在 RViz 里应该是往前转——如果 URDF

joint 轴方向与本实现的 `velocity` 符号约定相反，轮子会倒转。

下一步：起 RViz，发一段 `+x` 方向的 `/cmd_vel`，肉眼确认轮子转向。 **方向错了 `hz` 一样是 49.9**，只有看图能发现。

下一步行动清单

优先级	事项
高	RViz 目视验证轮子转向 + 走圆弧看轨迹方向
高	真机接入前处理 uint16 回绕
中	加 `deadband_pulses` 死区
中	接 `/scan` + AMCL，观察实际漂移量是否与量化界吻合
低	补齐 B1 建议的 `base_frame`/`odom_frame`/`publish_rate`/`stale_timeout` 参数化

七、可复用的验证资产

本次为验证实现，新增了两个测试脚本（都在 `无硬件自测/`）：

脚本	作用	结果
`crosscheck_bridge_functions.py`	6 个纯函数 vs `test_protocol.py` 逐函数对拍 + 20 万组随机穷举	61 / 61

脚本	作用	结果
`sim_integration_check.py`	用 `DiffDriveSim` 真实模型驱动积分逻辑	19 / 19
`test_protocol.py`（原有）	协议与运动学单测回归	44 / 44

合计 124 条。

技术要点：`chassis_bridge.py` 顶部 import 了 `rclpy`/`serial`/`tf2_ros`， 无 ROS2 环境下直接 import 会失败。对拍脚本用 **AST 只摘模块级的 常量赋值（`Assign`）和函数定义（`FunctionDef`）**，在独立命名空间 exec： 既能测纯逻辑，又不碰 ROS2。

★ 坑：**必须把 `Assign` 一起摘**，否则 `ODOM_FIELDS` 等常量未定义会 `NameError`。

附：一句话总结

这次交付的核心不是「会解析协议」，而是「知道并发模型怎么摆、异常怎么分层」 ——前者有标准答案，后者才是 S20 的考点。

三条硬要求的本质：

- **R1** 是「别让阻塞调用进 spin 线程」
- **R2** 是「让发布节奏与被接收节奏解耦」
- **R3** 是「按故障的频繁程度分级决定日志噪声」

而这次踩的坑（install 落后、symlink 结构变化）共同指向一条工程直觉： **永远用询问代替假设**——问 Python 实际加载了什么，而不是猜路径。